

Proyecto Blockchain: Gobernanza y sistemas de votación

Carmen Miguel
Mariana Cucicea

Índice

1. Código	2
1.1. QuadraticVoting	2
1.1.1. Atributos	11
1.1.2. Constuctor	12
1.1.3. openVoting	12
1.1.4. addParticipant	13
1.1.5. removeParticipant	14
1.1.6. addProposal	14
1.1.7. reclamarTokensCanceladas	15
1.1.8. cancelProposal	16
1.1.9. buyTokens	16
1.1.10. sellTokens	17
1.1.11. getters	18
1.1.12. stake	19
1.1.13. checkAndExecuteProposal	20
1.1.14. withdrawFromProposal	21
1.1.15. closeVoting y reiniciarVotacion	22
1.1.16. reclamarTokens	23
1.1.17. ejecutarPropuestaSignaling	23
1.2. VoteToken	24
1.3. IExecutableProposal	25
2. Pruebas	25
2.1. Ejecución de cada función	25
2.1.1. Creación del contrato QuadraticVoting	25
2.1.2. Apertura de la votación	27
2.1.3. Añadir un participante	27
2.1.4. Eliminar un participante	30
2.1.5. Añadir una propuesta	30
2.1.6. Obtener propuestas de financiación pendientes	32
2.1.7. Obtener propuestas de signaling	32
2.1.8. Sell tokens	32
2.1.9. Buy tokens	33
2.1.10. Cancelar propuesta	34
2.1.11. Stake	34
2.1.12. Obtener propuestas aprobadas	37
2.1.13. Obtener propuestas canceladas	38
2.1.14. Reclamar tokens de propuestas canceladas	38
2.1.15. Cerrar la votación	38
2.1.16. Ejecutar propuestas signaling	39
2.1.17. Retirar tokens de una propuesta	39
2.1.18. Reclamar tokens	39

1. Código

1.1. QuadraticVoting

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.0;
3
4 import "contracts/VoteToken.sol";
5 import "contracts/IExecutableProposal.sol";
6
7 contract QuadraticVoting{
8
9     //ATRIBUTOS
10    uint public _precioToken;
11    uint public _maxTokens;
12    uint public _numTokens;
13    uint public _presupuesto;
14    uint public _contadorIDS;
15    uint public _estadoVotacion; // (0: cerrada, 1:abierta, 2:estado intermedio
    para las funciones del patron pull over push)
16    uint public _ronda; //para el patron, para que sea posible recuperar tokens de
    rondas ya cerradas etc
17    VoteToken _contratoERC20;
18    address public _propietario;
19    uint public _numParticipantes;
20    uint public _numPendingProposals;
21
22    struct Propuesta{
23        uint presupuesto;
24        uint numVotos;
25        uint numTokens;
26        uint posicion_array;//para saber en que posicion, del array de propuestas
        correspondiente, se encuentra
27        uint ronda; //a que ronda pertenece
28        address contrato;
29        address creador;
30        bool aprobada; // true: aprobada o false: pendiente
31        bool cancelada;//para que se pueda reclamar los tokens luego en la funcion
        y no haga el bucle
32        string titulo;
33        string descripcion;
34    }
35
36    mapping(uint=>mapping(address => uint) )votoPorVotante; //indexado por
        idVotante cuantos votos ha dado cada participante concreto
37
38    mapping (address => bool) public _inscritos; // relaciona a la direccion con
        si esta activo
39    mapping(uint=> Propuesta) public _propuestas; // relaciona el id de una
        propuesta con su info
40    //las propuestas se separan en arrays dependiendo de su estado o tipo
41    uint[] public _propuestasAprobadas;
42    uint[] public _propuestasSignaling;
43    uint[] public _propuestasFinanciacion;
44    uint[] public _propuestasCanceladas; //consultar las propuestas canceladas
45
46
47
48    //MODIFICADORES
49
```

```

50 //verificar que la funcion la ejecuta solo el propietario
51 modifier onlyOwner() {
52     require (_propietario == msg.sender, "No eres el propietario del contrato"
53     );
54     -;
55 }
56 //verificar que la funcion se ejecuta solo con la votacion abierta
57 modifier estaAbierta() {
58     require (_estadoVotacion == 1, "Votacion no abierta");
59     -;
60 }
61 //verificar que la funcion se ejecuta solo con la votacion cerrada
62 modifier estaCerrada() {
63     require (_estadoVotacion == 0, "Votacion no cerrada");
64     -;
65 }
66 //verificar que la funcion se ejecuta solo con el estado intermedio para el
67 // patron pull over push
68 modifier estadoIntermedio() {
69     require (_estadoVotacion == 2, "El estado de la votacion no lo permite");
70     -;
71 }
72 //verificar que la direccion esta activa en la votacion
73 modifier estaActivo() {
74     require (_inscritos[msg.sender] == true, "Direccion no activa");
75     -;
76 }
77 //FUNCIONES
78 constructor (uint precioToken, uint maxTokens){
79     _precioToken = precioToken;
80     _maxTokens = maxTokens;
81     _numTokens=0;
82     _presupuesto=0;
83     _contadorIDS = 0;
84     _propietario = msg.sender;
85     _estadoVotacion = 0; //cerrada
86     _ronda =0;
87     _numParticipantes=0;
88     _numPendingProposals=0;
89     _contratoERC20 = new VoteToken(address(this),"VoteToken", "VTK"); //
90     // creacion del contrato de los tokens
91 }
92 //Apertura de la votacion, solo por parte del propietario cuando la votacion
93 // esta cerrada
94 function openVoting () public payable onlyOwner() estaCerrada(){
95     _estadoVotacion = 1; //se cambia la votacion a abierta
96     _presupuesto = msg.value; //se debe transferir el presupuesto inicial
97     _ronda++; //se incrementa la ronda por cada ronda nueva de la votacion (
98     // que va desde 1)
99     _numPendingProposals = 0; //se reinicia al iniciar una nueva (propuestas
100     // pendientes de esta ronda)
101 }
102

```

```

103 //Inscripcion a la votacion en cualquier momento y transfieren ether para los
    tokens
104 function addParticipant () public payable{
105     //se comprueban restricciones
106     require(!_inscritos[msg.sender]==false, "Ya estas activo");
107     require(msg.value >= _precioToken, "Debes enviar al menos el precio de un
        token");
108
109     //se crea el numero de tokens correspondiente
110     uint tokensComprados = msg.value / _precioToken; //toda division trunca el
        resultado
111     //no hay problema con overflow por la version del compilador
112     require(_numTokens + tokensComprados <= _maxTokens, "Supera el maximo de
        tokens");
113
114     //se crean tokens directamente desde este contrato (que es admin del token
        )
115     _contratoERC20.createToken(msg.sender, tokensComprados);
116     _numTokens += tokensComprados;
117
118     //activar participante en las estructuras
119     _inscritos[msg.sender] = true;
120     _numParticipantes++;
121
122     //devolver lo que sobra despues para evitar reentrada
123     uint resto = msg.value % _precioToken;
124     if (resto > 0) {
125         (bool success, ) = msg.sender.call{value: resto}("");
126         require(success, "Fallo al devolver el cambio");
127     }
128 }
129
130 //Eliminacion de una direccion de la votacion
131 function removeParticipant () public estaActivo(){
132
133     //gestion de estructuras
134     _inscritos[msg.sender] = false;
135     _numParticipantes--;
136 }
137
138 //Meter una propuesta en el sistema, solo cuando la direccion esta inscrita y
    la votacion abierta
139 function addProposal (string memory titulo, string memory descripcion, uint
    presupuesto, address contrato) public estaActivo() estaAbierta() returns (
    uint){
140
141     //comprobar que la propuesta implementa la interfaz
142     require(IERC165(contrato).supportsInterface(type(IExecutableProposal).
        interfaceId), "El contrato no implementa IExecutableProposal");
143
144     //se crea un nuevo id para la propuesta
145     _contadorIDS++;
146     uint id = _contadorIDS;
147
148     //posicion que pasa a ocupar en el array correspondiente
149     uint pos = 0;
150     if(presupuesto>0){
151         _propuestasFinanciacion.push(id);
152         pos = _propuestasFinanciacion.length - 1;
153         _numPendingProposals++;

```

```

154     }
155     else{
156         _propuestasSignaling.push(id);
157         pos = _propuestasSignaling.length-1;
158     }
159
160     _propuestas[id] = Propuesta({
161         presupuesto: presupuesto,
162         numVotos: 0,
163         numTokens: 0,
164         posicion_array : pos,
165         ronda: _ronda,
166         contrato: contrato,
167         creador : msg.sender,
168         aprobada: false,
169         cancelada: false,
170         titulo: titulo,
171         descripcion: descripcion
172     });
173
174
175     return id;
176 }
177
178 //reclamar los tokens de las propuestas que han sido canceladas (bajo demanda
179 //siguiendo el patron)
180 function reclamarTokensCanceladas(uint id) public estaActivo() {
181     Propuesta storage p = _propuestas[id];
182     require(p.cancelada, "Propuesta no cancelada");
183     require (p.aprobada == false, "Propuesta aprobada");
184
185     uint votos = votoPorVotante[id][msg.sender];
186     require(votos > 0, "No tienes votos en esta propuesta");
187
188     uint tokensADevolver = votos * votos;
189     votoPorVotante[id][msg.sender] = 0;
190     p.numVotos-=votos;
191     p.numTokens-=tokensADevolver;
192
193     bool success = _contratoERC20.transferFrom(address(this), msg.sender ,
194         tokensADevolver);
195     require(success, "Fallo al transferir tokens");
196
197     if(p.numVotos == 0) {
198         uint movida = _propuestasCanceladas[_propuestasCanceladas.length-1];
199         _propuestas[movida].posicion_array = p.posicion_array;
200         _propuestasCanceladas[p.posicion_array] = movida;
201         _propuestasCanceladas.pop();
202
203         delete _propuestas[id];
204     }
205 }
206
207 //Cancelar propuesta, solo con la votacion abierta por el creador de esta
208 function cancelProposal (uint id) public estaAbierta() estaActivo(){
209
210     //se hace una variable en storage para menor cantidad de accesos
211     Propuesta storage p = _propuestas[id];

```

```

212 //comprobar restricciones
213 require (p.creador == msg.sender, "Solo puede cancelar una propuesta su
    creador");
214 require(p.aprobada == false, "No se puede cancelar una propuesta aprobada"
    );
215 require(p.cancelada == false, "Propuesta ya cancelada");
216
217 p.cancelada = true; //se marca como cancelada y se evita el bucle
218
219 //en el array de las propuestas se elimina
220 if(p.presupuesto>0){
221     uint movida = _propuestasFinanciacion[_propuestasFinanciacion.length
        -1];
222     _propuestas[movida].posicion_array = p.posicion_array;
223     _propuestasFinanciacion[p.posicion_array] = movida;
224     _propuestasFinanciacion.pop();
225     _numPendingProposals--;
226
227 }
228 else{
229     uint movida = _propuestasSignaling[_propuestasSignaling.length-1];
230     _propuestas[movida].posicion_array = p.posicion_array;
231     _propuestasSignaling[p.posicion_array] = movida;
232     _propuestasSignaling.pop();
233 }
234
235
236 //a canceladas y no se borra del mapa
237 _propuestasCanceladas.push(id);
238 p.posicion_array = _propuestasCanceladas.length - 1;
239 }
240
241 //Compra de tokens para una direccion inscrita
242 function buyTokens () public payable estaActivo(){
243
244     require(msg.value >= _precioToken, "Al menos el precio de un token");
245
246     uint tokensComprados = msg.value / _precioToken; //toda division trunca el
        resultado
247     require(_numTokens + tokensComprados <= _maxTokens, "Supera el maximo de
        tokens");
248
249     _numTokens += tokensComprados;
250     _contratoERC20.createToken(msg.sender, tokensComprados);
251
252
253     //devolver resto
254     uint resto = msg.value % _precioToken;
255     if (resto > 0) {
256         (bool success, ) = msg.sender.call{value: resto}("");
257         require(success, "Fallo al devolver el cambio");
258     }
259 }
260
261 //Venta de tokens para una direccion inscrita
262 function sellTokens (uint devolver) public estaActivo(){
263     uint numtokens = _contratoERC20.balanceOf(msg.sender);
264     require(devolver <= numtokens, "No tienes suficientes tokens");
265
266     //calcular cantidad y quemarlos

```

```

267     uint cantidad = devolver * _precioToken;
268     _contratoERC20.burnToken(msg.sender, numtokens); //actualiza balance para
        que no haya reentrada
269     _numTokens -= devolver;
270     //enviar cantidad
271     (bool success, ) = msg.sender.call{value: cantidad}("");
272     require(success, "Error al enviar la cantidad");
273
274 }
275
276 //Direccion del contrato de los tokens para participantes
277 function getERC20 () view public estaActivo() returns (address){
278     return address(_contratoERC20);
279 }
280
281 //Array con ids de las propuestas pendientes, solo con la votacion abierta
282 function getPendingProposals () view public estaAbierta() returns (uint[]
    memory){
283     return _propuestasFinanciacion;
284 }
285
286 //Array con ids de las propuestas en aprobadas, solo con la votacion abierta
287 function getApprovedProposals () view public estaAbierta() returns (uint[]
    memory){
288     return _propuestasAprobadas;
289 }
290
291 //Array con ids de las propuestas en signaling, solo con la votacion abierta
292 function getSignalingProposals () view public estaAbierta() returns (uint[]
    memory){
293     return _propuestasSignaling;
294 }
295
296 //Array con ids de las propuestas canceladas
297 function getCanceledProposals () view public estaAbierta() returns (uint[]
    memory){
298     return _propuestasCanceladas;
299 }
300
301 //Obtener informacion de una propuesta
302 function getProposalInfo (uint id) view public estaAbierta() returns (
    Propuesta memory){
303     Propuesta memory info = _propuestas[id];
304     return info;
305 }
306
307 //Realiza el voto de quien ejecuta la funcion, debe estar inscrito y la
    votacion abierta
308 function stake (uint id, uint nuevosVotos) public estaActivo() estaAbierta(){
309     require(nuevosVotos > 0, "Al menos un voto");
310
311     Propuesta storage p = _propuestas[id];
312     require(!p.aprobada, "Propuesta ya aprobada");
313     require(!p.cancelada, "Propuesta cancelada");
314
315     //debe pertenecer a esta ronda para poder votarla
316     require(p.ronda == _ronda, "La propuesta no pertenece a esta ronda");
317
318     //se comprueban los votos a esta propuesta por parte del participante
319     uint votosPrevios = votoPorVotante[id][msg.sender];

```



```

320     uint totalVotos = votosPrevios + nuevosVotos;
321
322     //calculo cuadratico del coste adicional
323     uint costeAntes = votosPrevios * votosPrevios;
324     uint costeDespues = totalVotos * totalVotos;
325     uint costeExtra = costeDespues - costeAntes; //tokens necesarios para
        realizar los votos
326
327     //verificar allowance (el approve lo hizo el usuario antes de llamar a
        esta funcion)
328     require(_contratoERC20.allowance(msg.sender, address(this)) >= costeExtra,
        "Aprobacion insuficiente");
329
330     bool success = _contratoERC20.transferFrom(msg.sender, address(this),
        costeExtra);
331     require(success, "Fallo al transferir tokens");
332
333     //registrar los votos y sumar al total de la propuesta
334     p.numTokens += costeExtra;
335     p.numVotos += nuevosVotos;
336     votoPorVotante[id][msg.sender] += nuevosVotos;
337
338     //comprobar si se puede aprobar la propuesta
339     if(p.presupuesto>0){ //solo las de financiacion se aprueban aqui, las
        signalling al finalizar
340         _checkAndExecuteProposal(id);
341     }
342 }
343
344 //Retirar los votos del participante de cierta propuesta
345 function withdrawFromProposal (uint id, uint cantidad) public estaActivo()
    estaAbierta() {
346
347     Propuesta storage p = _propuestas[id];
348     //solo puede retirar votos de una propuesta no aprobada y sus propios
        votos
349     require(!p.aprobada, "Propuesta ya aprobada");
350     uint votos = votoPorVotante[id][msg.sender];
351     require(votos >= cantidad, "No tienes votos suficientes en esta propuesta"
        );
352
353     //debe pertenecer a esta ronda para poder retirar votos
354     require(p.ronda == _ronda, "La propuesta no pertenece a esta ronda");
355
356     //cantidad a devolver siguiendo la logica cuadratica
357     uint votosRestantes = votos - cantidad;
358     uint tokensADevolver = (votos * votos) - (votosRestantes * votosRestantes)
        ;
359
360     //actualizar estado
361     votoPorVotante[id][msg.sender] -= cantidad;
362     p.numVotos -= cantidad;
363     p.numTokens -= tokensADevolver;
364
365     //devolver tokens
366     bool success = _contratoERC20.transferFrom(address(this), msg.sender ,
        tokensADevolver);
367     require(success, "Fallo al transferir tokens");
368 }
369

```

```

370
371 //se usa para calcular el umbral para que una propuesta se acepte
372 function _calcularUmbral(uint id) internal view returns (uint) {
373
374     if (_presupuesto == 0) return type(uint).max; // para evitar division por
375         0
376
377     Propuesta storage p = _propuestas[id];
378
379     uint umbral = ((2e17*_presupuesto + p.presupuesto * 1e18)*
380         _numParticipantes / _presupuesto) / 1e18 + _numPendingProposals;
381     return umbral;
382 }
383
384 //Comprueba que la propuesta se debe aceptar y ejecuta executeProposal
385 function _checkAndExecuteProposal (uint id) internal {
386     Propuesta storage p = _propuestas[id];
387     //se calcula el umbral con la formula dada en el enunciado
388     uint umbral = _calcularUmbral(id);
389
390     //se comprueba si se puede aprobar
391     if (p.numVotos > umbral && p.presupuesto <= _presupuesto) {
392         p.aprobada = true;
393         _propuestasAprobadas.push(id);
394         uint movida = _propuestasFinanciacion[_propuestasFinanciacion.length
395             -1];
396         _propuestas[movida].posicion_array = p.posicion_array;
397         _propuestasFinanciacion[p.posicion_array] = movida;
398         _propuestasFinanciacion.pop();
399         _presupuesto += p.numTokens*_precioToken;
400         _presupuesto -= p.presupuesto;
401         _numPendingProposals--;
402
403         //quemar los tokens
404         _contratoERC20.burnToken(address(this), p.numTokens);
405
406         //llamar a la funcion de ejecutar
407         (bool success, ) = p.contrato.call{value: p.presupuesto, gas: 100000}({
408             abi.encodeWithSignature("executeProposal(uint256,uint256,uint256)"
409                 , id, p.numVotos, p.numTokens)
410         });
411         require(success, "Fallo al ejecutar la propuesta");
412     }
413 }
414
415 //Cierre de la votacion solo por el propietario
416 function closeVoting () public onlyOwner() estaAbierta() {
417     _estadoVotacion = 2; //se pone en estado intermedio para que puedan
418     ejecutar las funciones
419 }
420
421 //Patron pull over push
422
423 function reiniciarVotacion() public onlyOwner() estadoIntermedio(){
424     if (_presupuesto > 0) {
425         uint restante = _presupuesto;

```

```

425         _presupuesto = 0; //antes para evitar reentrada
426         (bool success, ) = _propietario.call{value: restante}("");
427         require(success, "Error al devolver Ether");
428     }
429     _estadoVotacion = 0; //pasa a estar cerrado
430     _numPendingProposals = 0;
431     delete _propuestasAprobadas;
432     delete _propuestasFinanciacion;
433     delete _propuestasSignaling;
434 }
435
436 //reclamar los tokens cuando ha terminado la votacion (bajo demanda siguiendo
    el patron)
437 function reclamarTokens(uint id) public estaActivo(){
438     Propuesta storage p = _propuestas[id];
439     require(!p.aprobada, "Propuesta aprobada");
440
441     uint votos = votoPorVotante[id][msg.sender];
442     require(votos > 0, "No tienes votos en esta propuesta");
443
444     require(p.ronda < _ronda || (p.ronda == _ronda && _estadoVotacion ==2), "
        Propuesta de ronda no valida");
445
446     uint tokensADevolver = votos * votos;
447     votoPorVotante[id][msg.sender] = 0; //antes para evitar reentrada
448     p.numVotos -=votos;
449     p.numTokens -=tokensADevolver;
450
451     bool success = _contratoERC20.transferFrom(address(this), msg.sender,
        tokensADevolver);
452     require(success, "Fallo al transferir tokens");
453     if(p.numVotos == 0) delete _propuestas[id];
454 }
455
456 //ejecutar las propuestas signaling bajo demanda
457 function ejecutarPropuestaSignaling(uint id) public {
458     Propuesta storage p = _propuestas[id];
459     require(!p.aprobada, "Propuesta ya aprobada");
460     require(p.presupuesto == 0, "No es signaling");
461     require(p.creador == msg.sender, "Solo el creador puede ejecutarla");
462     require(p.ronda < _ronda || (p.ronda == _ronda && _estadoVotacion ==2), "
        Propuesta de ronda no valida");
463
464     p.aprobada = true;
465
466     //quemar los tokens
467     _contratoERC20.burnToken(address(this), p.numTokens);
468
469     (bool success, ) = p.contrato.call{gas: 100000}{
470         abi.encodeWithSignature("executeProposal(uint256,uint256,uint256)", id
            , p.numVotos, p.numTokens)
471     };
472
473     require(success, "Fallo al ejecutar signaling");
474
475 }
476
477 }

```

Listing 1: QuadraticVoting.sol

Se realiza una explicación de cada una de las funciones del contrato. Para ello, se divide la sección en subsecciones en función del número de funciones que hay para poder explorar cada detalle de la implementación en profundidad.

1.1.1. Atributos

```

1 //ATRIBUTOS
2     uint public _precioToken;
3     uint public _maxTokens;
4     uint public _numTokens;
5     uint public _presupuesto;
6     uint public _contadorIDS;
7     uint public _estadoVotacion; // (0: cerrada, 1:abierta, 2:estado intermedio
8     para las funciones del patron pull over push)
9     uint public _ronda; //para el patron, para que sea posible recuperar tokens de
10     rondas ya cerradas etc
11     VoteToken _contratoERC20;
12     address public _propietario;
13     uint public _numParticipantes;
14     uint public _numPendingProposals;
15
16     struct Propuesta{
17         uint presupuesto;
18         uint numVotos;
19         uint numTokens;
20         uint posicion_array;//para saber en que posicion, del array de propuestas
21         correspondiente, se encuentra
22         uint ronda; //a que ronda pertenece
23         address contrato;
24         address creador;
25         bool aprobada; // true: aprobada o false: pendiente
26         bool cancelada;//para que se pueda reclamar los tokens luego en la funcion
27         y no haga el bucle
28         string titulo;
29         string descripcion;
30     }
31
32     mapping(uint=>mapping(address => uint) )votoPorVotante; //indexado por
33     idVotante cuantos votos ha dado cada participante concreto
34
35     mapping (address => bool) public _inscritos; // relaciona a la direccion con
36     si esta activo
37
38     mapping(uint=> Propuesta) public _propuestas; // relaciona el id de una
39     propuesta con su info
40
41     //las propuestas se separan en arrays dependiendo de su estado o tipo
42     uint[] public _propuestasAprobadas;
43     uint[] public _propuestasSignaling;
44     uint[] public _propuestasFinanciacion;
45     uint[] public _propuestasCanceladas;

```

Listing 2: Atributos del contrato

Aparecen los atributos que controlan aspectos que se pedían en el enunciado: el valor en weis de cada token, el número máximo de tokens que permitirá el sistema de votación, el número de tokens que hay en el momento en el sistema, el presupuesto que se tiene, la dirección del contrato que gestiona los tokens, la dirección del propietario del contrato, el número de participantes inscritos activos y el número de propuestas pendientes.

Además, se declara la variable ronda que hace referencia a la ronda que pertenece la votación que se está llevando a cabo. Esta variable ha sido implementada para poder desarrollar el patrón opcional pull over

push y hacer ciertas tareas bajo demanda de los clientes evitando así bucles en ciertas funciones.

Para la gestión de propuestas, como cada una se identifica con un id se declara la variable contadorIDS. Además, se crea una estructura con la información correspondiente a cada propuesta: su título, descripción, presupuesto necesario al aprobarse, la dirección del contrato que representa la propuesta, la dirección de su creador, el número de votos que ha recibido hasta el momento, el número de tokens correspondientes a los votos recibidos, la posición de la propuesta en el array correspondiente según su tipo, la variable aprobada que indica si la propuesta ha sido aprobada o no, el número de ronda al que pertenece la propuesta y la variable que indica si la propuesta ha sido cancelada. Todos estos campos han sido reordenados para mejorar la ocupación de espacio en storage y que los booleanos se compriman en el mismo slot con una variable de tipo address. Se declara un mapping de mappings que relaciona el id de la propuesta con la dirección de un votante y a esta con el número de votos que ha emitido a la propuesta correspondiente.

Para la gestión de los participantes, se declara un mapping (inscritos) que relaciona una dirección con un booleano que indica si está activo o no.

Se declaran arrays dinámicos para tipo (signaling o financiación que están pendientes) o estado de una propuesta (aprobada o cancelada).

1.1.2. Constructor

```
1 constructor (uint precioToken, uint maxTokens){
2     _precioToken = precioToken;
3     _maxTokens = maxTokens;
4     _numTokens=0;
5     _presupuesto=0;
6     _contadorIDS = 0;
7     _propietario = msg.sender;
8     _estadoVotacion = 0; //cerrada
9     _ronda =0;
10    _numParticipantes=0;
11    _numPendingProposals=0;
12    _contratoERC20 = new VoteToken(address(this),"VoteToken", "VTK"); //
        creacion del contrato de los tokens
13 }
```

Listing 3: Función constructor

En el constructor del contrato se fija el precio de cada token y el número máximo de estos que puede haber en el sistema. Se inicializan las variables descritas anteriormente según se necesita y se crea el contrato que gestiona los tokens ERC20 del sistema de votación.

1.1.3. openVoting

```
1 //Apertura de la votacion, solo por parte del propietario cuando la votacion esta
    cerrada
2     function openVoting () public payable onlyOwner() estaCerrada(){
3         _estadoVotacion = 1; //se cambia la votacion a abierta
4         _presupuesto = msg.value; //se debe transferir el presupuesto inicial
5         _ronda++; //se incrementa la ronda por cada ronda nueva
6         _numPendingProposals = 0; //se reinicia al iniciar una nueva (propuestas
            pendientes de esta ronda)
7     }
```

Listing 4: Función openVoting

La función está implementada para usarse cada vez que se quiera iniciar una ronda de votación. Solo la puede utilizar el propietario del contrato cuando se ha cerrado la anterior ronda y es de tipo payable para que se le pase el presupuesto de esta nueva ronda. Se establece la votación como abierta y se incrementa el número de ronda actual.

1.1.4. addParticipant

```
1 //Inscripcion a la votacion en cualquier momento y transfieren ether para los
  tokens
2   function addParticipant () public payable{
3
4       //se comprueban restricciones
5       require(!_inscritos[msg.sender]==false, "Ya estas activo");
6       require(msg.value >= _precioToken, "Debes enviar al menos el precio de un
          token");
7
8       //se crea el numero de tokens correspondiente
9       uint tokensComprados = msg.value / _precioToken; //toda division trunca el
          resultado
10
11      //no hay problema con overflow por la version del compilador
12      require(_numTokens + tokensComprados <= _maxTokens, "Supera el maximo de
          tokens");
13
14      //se crean tokens directamente desde este contrato (que es admin del token
          )
15      _contratoERC20.createToken(msg.sender, tokensComprados);
16      _numTokens += tokensComprados;
17
18      //activar participante en las estructuras
19      _inscritos[msg.sender] = true;
20      _numParticipantes++;
21
22      //devolver lo que sobra despues para evitar reentrada
23      uint resto = msg.value % _precioToken;
24      if (resto > 0) {
25
26          (bool success, ) = msg.sender.call{value: resto}("");
27          require(success, "Fallo al devolver el cambio");
28
29      }
30
31  }
```

Listing 5: Función addParticipant

La función sirve para añadir un participante y puede usarse en cualquier momento transfiriendo cierta cantidad de ether que debe ser como mínimo el valor de un token. Se encarga de gestionar las estructuras con este nuevo participante y de crear la cantidad de tokens correspondientes al valor transferido por el cliente usando el contrato de gestión de tokens (debido a la versión del compilador no hay problemas con el overflow en ciertas operaciones aritméticas necesarias para las comprobaciones). En caso de que el valor transferido sea superior a la compra de un número de tokens se devuelve el sobrante al cliente con una transferencia de Ether plana, lo cual es susceptible de sufrir ataques de reentrada al perder el control de la ejecución.

No se pueden dar ataques con una llamada a esta misma función por haber modificado la variable booleana que indica que está el participante activo antes del call y comprobar que está a falso en el require del principio. En caso de que acceda a cualquier otra función del contrato, no puede sacar beneficio ni bloquear el sistema ya que estaría usando las funciones como si fuese un participante activo más.

1.1.5. removeParticipant

```
1 //Eliminacion de una direccion de la votacion
2     function removeParticipant () public estaInscrito(){
3         //gestion de estructuras
4         _inscritos[msg.sender] = false;
5         _numParticipantes--;
6     }
```

Listing 6: Función removeParticipant

Esta función actúa como complementaria a la anterior, marcando como inactivo a un participante en el sistema. Gracias a las estructuras y atributos usados se ha conseguido gestionar esto simplemente modificando la variable activo de cierto participante en el mapping de inscritos, de manera que pueda volver a inscribirse sin perder su información anterior.

1.1.6. addProposal

```
1 //Meter una propuesta, solo cuando la direccion esta inscrita y la votacion
  abierta
2     //comprobar que la propuesta implementa la interfaz
3     require(IERC165(contrato).supportsInterface(type(IExecutableProposal).
      interfaceId),"El contrato no implementa IExecutableProposal");
4
5     //se crea un nuevo id para la propuesta
6     _contadorIDS++;
7     uint id = _contadorIDS;
8
9     //posicion que pasa a ocupar en el array correspondiente
10    uint pos = 0;
11    if(presupuesto>0){
12        _propuestasFinanciacion.push(id);
13        pos = _propuestasFinanciacion.length - 1;
14        _numPendingProposals++;
15    }
16    else{
17        _propuestasSignaling.push(id);
18        pos = _propuestasSignaling.length-1;
19    }
20
21    _propuestas[id] = Propuesta({
22        presupuesto: presupuesto,
23        numVotos: 0,
24        numTokens: 0,
25        posicion_array : pos,
26        ronda: _ronda,
27        contrato: contrato,
28        creador : msg.sender,
29        aprobada: false,
30        cancelada: false,
31        titulo: titulo,
32        descripcion: descripcion
33    });
34
35    return id;
36 }
```

Listing 7: Función addProposal

La función se usa para añadir una propuesta al sistema de votación, lo cual solo puede hacerlo una dirección inscrita en el sistema con la votación abierta. Recibe toda la información relativa a la propuesta y comprueba que la dirección del contrato de la propuesta implementa la interfaz necesaria. Se encarga de asignar un id y una posición en el array correspondiente, así como de crear la estructura que guardará toda la información de la propuesta.

1.1.7. reclamarTokensCanceladas

```

1 //reclamar los tokens de las propuestas que han sido canceladas (bajo demanda
  siguiendo el patron)
2 function reclamarTokensCanceladas(uint id) public estaActivo() {
3     Propuesta storage p = _propuestas[id];
4     require(p.cancelada, "Propuesta no cancelada");
5     require (p.aprobada == false, "Propuesta aprobada");
6
7     uint votos = votoPorVotante[id][msg.sender];
8     require(votos > 0, "No tienes votos en esta propuesta");
9
10    uint tokensADevolver = votos * votos;
11    votoPorVotante[id][msg.sender] = 0;
12    p.numVotos -= votos;
13    p.numTokens -= tokensADevolver;
14
15    bool success = _contratoERC20.transferFrom(address(this), msg.sender,
      tokensADevolver);
16    require(success, "Fallo al transferir tokens");
17
18    if(p.numVotos == 0) {
19        uint movida = _propuestasCanceladas[_propuestasCanceladas.length-1];
20        //*****
21        _propuestas[movida].posicion_array = p.posicion_array;
22        _propuestasCanceladas[p.posicion_array] = movida;
23        _propuestasCanceladas.pop();
24
25        delete _propuestas[id];
26    }
27 }

```

Listing 8: Función reclamarTokensCanceladas

Es una de las funciones implementadas para evitar bucles con el patrón pull-over-push. Con ella, los participantes activos puedes reclamar los tokens usados en la votación de propuestas canceladas, en lugar de devolver los tokens automáticamente una vez que una propuesta se cancela (para lo que era necesario un bucle que iterase en el número de votantes de la propuesta).

Se encarga de recibir el id de la propuesta cancelada de la cual se quieren extraer votos y comprueba si la dirección que la ejecuta tiene votos en ella y si está en efecto cancelada y no ha sido aprobada. Una vez hechas las comprobaciones, calcula los tokens que deben devolverse, siguiendo el sistema cuadrático, con el dato de los votos emitidos a la propuesta por parte del participante que ejecuta la función. Así, se actualizan los datos de la propuesta (no sufren underflow por la versión del compilador y además porque el sistema se encarga de modificar los datos correctamente tal que el número de votos totales a cierta propuesta siempre sera mayor que los hechos a una propuesta por parte de un participante, de igual manera ocurre con el número de tokens), se devuelven la propiedad de los tokens al participante y, en caso de que no quede ningún participante por reclamar a esta propuesta, se borra del array, siguiendo la política de que el último ocupe el hueco que se deja y del mapping para liberar espacio.

1.1.8. cancelProposal

```
1 //Cancelar propuesta, solo con la votacion abierta por el creador de esta
2   function cancelProposal (uint id) public estaAbierta() estaActivo(){
3
4       //se hace una variable en storage para menor cantidad de accesos
5       Propuesta storage p = _propuestas[id];
6
7       //comprobar restricciones
8       require (p.creador == msg.sender, "Solo puede cancelar una propuesta su
9           creador");
10      require(p.aprobada == false, "No se puede cancelar una propuesta aprobada"
11          );
12      require(p.cancelada == false, "Propuesta ya cancelada");
13
14      p.cancelada = true; //se marca como cancelada y se evita el bucle
15
16      //en el array de las propuestas se elimina
17      if(p.presupuesto>0){
18          uint movida = _propuestasFinanciacion[_propuestasFinanciacion.length
19              -1];
20          _propuestas[movida].posicion_array = p.posicion_array;
21          _propuestasFinanciacion[p.posicion_array] = movida;
22          _propuestasFinanciacion.pop();
23          _numPendingProposals--;
24      }
25      else{
26          uint movida = _propuestasSignaling[_propuestasSignaling.length-1];
27          _propuestas[movida].posicion_array = p.posicion_array;
28          _propuestasSignaling[p.posicion_array] = movida;
29          _propuestasSignaling.pop();
30      }
31
32      //a canceladas y no se borra del mapa
33      _propuestasCanceladas.push(id);
34      p.posicion_array = _propuestasCanceladas.length - 1;
35  }
```

Listing 9: Función cancelProposal

La función permite cancelar una propuesta al creador de esta (estando activo) mientras la votación está abierta y la propuesta no ha sido aprobada o cancelada previamente. Se encarga de distinguir según su tipo en el array que se encuentra y lo saca de este evitando la fragmentación del array poniendo en la posición que queda libre la última propuesta añadida. Por último, se añade al array de propuestas canceladas y se modifica el campo de posicion en el array a su posición en el nuevo array de propuestas canceladas. No se puede borrar su entrada en el mapping para que los participantes puedan reclamar los tokens en su momento.

1.1.9. buyTokens

```
1 //Compra de tokens para una direccion inscrita
2   function buyTokens () public payable estaActivo(){
3
4       require(msg.value >= _precioToken, "Al menos el precio de un token");
5
6       uint tokensComprados = msg.value / _precioToken; //toda division trunca el
           resultado
```

```

7      require(_numTokens + tokensComprados <= _maxTokens, "Supera el maximo de
      tokens");
8
9      _numTokens += tokensComprados;
10     _contratoERC20.createToken(msg.sender, tokensComprados);
11
12
13     //devolver resto
14     uint resto = msg.value % _precioToken;
15     if (resto > 0) {
16         (bool success, ) = msg.sender.call{value: resto}("");
17         require(success, "Fallo al devolver el cambio");
18     }
19 }

```

Listing 10: Función buyTokens

Permite comprar tokens a participantes inscritos en el sistema de votación, comprobando que la cantidad transferida es como mínimo el valor de un token y no se sobrepasa el número máximo de tokens permitidos en el sistema (la suma no puede sufrir overflow por la versión del compilador). Con las comprobaciones hechas, se crea el número de tokens correspondiente haciendo uso del contrato de gestión de tokens y se devuelve al participante la cantidad de dinero sobrante, si hubiese.

Esta última transferencia plana provoca una pérdida del control de la ejecución por parte del contrato. Por ello, es necesario estudiar las posibles vulnerabilidades que puede presentar. Al modificar el número de tokens previo a la llamada y necesitar un valor en la transacción no podrá obtener beneficio alguno de llamar varias veces a buyTokens, de hecho, modificar la variable numTokens antes del call y comprobar que se llegue al máximo hará que eventualmente las llamadas deban parar. No puede sacar rédito de hacer múltiples llamadas a otras funciones desde el call de esta función.

1.1.10. sellTokens

```

1 //Venta de tokens para una direccion inscrita
2 function sellTokens (uint devolver) public estaActivo(){
3     uint numtokens = _contratoERC20.balanceOf(msg.sender);
4     require(devolver <= numtokens, "No tienes suficientes tokens");
5
6     //calcular cantidad y quemarlos
7     uint cantidad = devolver * _precioToken;
8     _contratoERC20.burnToken(msg.sender, numtokens); //actualiza balance para
9     que no haya reentrada
10    _numTokens -= devolver;
11    //enviar cantidad
12    (bool success, ) = msg.sender.call{value: cantidad}("");
13    require(success, "Error al enviar la cantidad");
14 }

```

Listing 11: Función sellTokens

Función complementaria a la anterior: permite a direcciones inscritas vender un número de tokens. Comprueba que el participante tiene al menos el número de tokens que quiere vender, calcula la cantidad a devolver y usa el contrato de gestión de tokens para eliminarlos antes de devolverle la cantidad correspondiente al participante.

Esta última transferencia plana puede hacer que el contrato se vea expuesto a ciertos ataques pero al llamar a la función burn antes del call, esta modifica el balance de la dirección del participante lo que hace que eventualmente se quede sin tokens en su balance y se impida la ejecución con el require del inicio de

la función. Por esta misma razón, no puede hacer uso de los tokens que ya ha decidido quemar: en caso de querer usarlos para votar la función `transferFrom` se asegurará de que tenga los tokens necesarios para emitir cierto número de votos y, en caso de no tenerlos, no se producirán cambios en el estado del sistema.

1.1.11. getters

```
1 //Direccion del contrato de los tokens para participantes
2 function getERC20 () view public estaInscrito() returns (address){
3     return address(_contratoERC20);
4 }
5
6 //Array con ids de las propuestas pendientes, solo con la votacion abierta
7 function getPendingProposals () view public estaAbierta() returns (uint[]
8     memory){
9     return _propuestasFinanciacion;
10 }
11
12 //Array con ids de las propuestas en aprobadas, solo con la votacion abierta
13 function getApprovedProposals () view public estaAbierta() returns (uint[]
14     memory){
15     return _propuestasAprobadas;
16 }
17
18 //Array con ids de las propuestas en signaling, solo con la votacion abierta
19 function getSignalingProposals () view public estaAbierta() returns (uint[]
20     memory){
21     return _propuestasSignaling;
22 }
23
24 //Array con ids de las propuestas canceladas
25 function getCanceledProposals () view public estaAbierta() returns (uint[]
26     memory){
27     return _propuestasCanceladas;
28 }
29
30 //Obtener informacion de una propuesta
31 function getProposalInfo (uint id) view public estaAbierta() returns (
32     Propuesta memory){
33     Propuesta memory info = _propuestas[id];
34     return info;
35 }
```

Listing 12: Funciones getters

Son funciones que permiten acceder a cierta información del sistema de votación:

- Dirección del contrato de gestión de tokens.
- Array de las propuestas de financiación pendientes de la ronda.
- Array de las propuestas aprobadas en la ronda.
- Array de las propuestas signaling de la ronda.
- Array de las propuestas canceladas: funciona como un histórico (todas las propuestas que han sido canceladas en cualquier ronda ya que pueden reclamarse los tokens en cualquier momento una vez cancelada la propuesta). El array puede llegar un momento en el que sea muy grande pero al ser una función `view` y no implicar una transacción no supone un coste en gas.

- Información sobre una propuesta dado su id. Por esta función ha sido necesario crear un mapping de mappings (votoPorVotante) sacando así el mapping del struct Propuesta para que fuese posible que un participante consultase información relevante de cierta propuesta.

1.1.12. stake

```

1 //Realiza el voto de quien ejecuta la funcion, debe estar inscrito y la votacion
  abierta
2 function stake (uint id, uint nuevosVotos) public estaActivo() estaAbierta(){
3     require(nuevosVotos > 0, "Al menos un voto");
4
5     Propuesta storage p = _propuestas[id];
6     require(!p.aprobada, "Propuesta ya aprobada");
7     require(!p.cancelada, "Propuesta cancelada");
8
9     //debe pertenecer a esta ronda para poder votarla
10    require(p.ronda == _ronda, "La propuesta no pertenece a esta ronda");
11
12    //se comprueban los votos a esta propuesta por parte del participante
13    uint votosPrevios = votoPorVotante[id][msg.sender];
14    uint totalVotos = votosPrevios + nuevosVotos;
15
16    //calcula cuadrático del coste adicional
17    uint costeAntes = votosPrevios * votosPrevios;
18    uint costeDespues = totalVotos * totalVotos;
19    uint costeExtra = costeDespues - costeAntes; //tokens necesarios para
      realizar los votos
20
21    //verificar allowance (el approve lo hizo el usuario antes de llamar a
      esta funcion)
22    require(_contratoERC20.allowance(msg.sender, address(this)) >= costeExtra,
      "Aprobacion insuficiente");
23
24    bool success = _contratoERC20.transferFrom(msg.sender, address(this),
      costeExtra);
25    require(success, "Fallo al transferir tokens");
26
27    //registrar los votos y sumar al total de la propuesta
28    p.numTokens += costeExtra;
29    p.numVotos += nuevosVotos;
30    votoPorVotante[id][msg.sender] += nuevosVotos;
31
32    //comprobar si se puede aprobar la propuesta
33    if(p.presupuesto>0){ //solo las de financiación se aprueban aqui, las
      signalling al finalizar
34        _checkAndExecuteProposal(id);
35    }
36 }

```

Listing 13: Función stake

Permite a un participante inscrito, estando la votación abierta, votar alguna de las propuestas de la ronda actual que están pendientes (ni aprobada ni cancelada) dado su id. La función calcula los tokens necesarios para emitir un voto teniendo en cuenta los votos emitidos a esta propuesta por parte del participante y el sistema cuadrático que se utiliza. Una vez hecho el cálculo sabiendo que no hay peligro de overflow o underflow por el compilador con el que se trabaja, se comprueba que el participante ha permitido al contrato QuadraticVoting operar con el número de tokens necesarios con el contrato de gestión de tokens y la función allowance. Para ello, el participante habrá tenido que hacer de forma externa al contrato una llamada a approve permitiendo la transferencia de la propiedad de cierto número de tokens a QuadraticVoting. Si se

ha permitido, se transfiere el número de tokens al contrato del sistema de votación y se registran los votos en la propuesta. Por último, si se trata de una propuesta de financiación se llama a la función que se encarga de comprobar si se puede aprobar.

1.1.13. checkAndExecuteProposal

```
1 //se usa para calcular el umbral para que una propuesta se acepte
2 function _calcularUmbral(uint id) internal view returns (uint) {
3
4     if (_presupuesto == 0) return type(uint).max; // para evitar division por
5         0
6
7     Propuesta storage p = _propuestas[id];
8
9     uint umbral = ((2e17*_presupuesto + p.presupuesto * 1e18)*
10         _numParticipantes / _presupuesto) / 1e18 + _numPendingProposals;
11     return umbral;
12 }
13
14 //Comprueba que la propuesta se debe aceptar y ejecuta executeProposal
15 function _checkAndExecuteProposal (uint id) internal {
16     Propuesta storage p = _propuestas[id];
17
18     //se calcula el umbral con la formula dada en el enunciado
19     uint umbral = _calcularUmbral(id);
20
21     //se comprueba si se puede aprobar
22     if (p.numVotos > umbral && p.presupuesto<=_presupuesto) {
23         p.aprobada = true;
24         _propuestasAprobadas.push(id);
25         uint movida = _propuestasFinanciacion[_propuestasFinanciacion.length
26             -1];
27         _propuestas[movida].posicion_array = p.posicion_array;
28         _propuestasFinanciacion[p.posicion_array] = movida;
29         _propuestasFinanciacion.pop();
30         _presupuesto += p.numTokens*_precioToken;
31         _presupuesto -= p.presupuesto;
32         _numPendingProposals--;
33
34         //quemar los tokens
35         _contratoERC20.burnToken(address(this), p.numTokens);
36
37         //llamar a la funcion de ejecutar
38         (bool success, ) = p.contrato.call{value: p.presupuesto, gas: 100000}({
39             abi.encodeWithSignature("executeProposal(uint256,uint256,uint256)"
40                 , id, p.numVotos, p.numTokens)
41         });
42         require(success, "Fallo al ejecutar la propuesta");
43     }
```

Listing 14: Función checkAndExecuteProposal

Función interna a la que se llama desde stake para comprobar si una función puede ser aprobada y, en caso de que sí, llamar a la función de ejecución asociada a la propuesta.

Se utiliza una función interna que calcula el umbral de votos que debe superar para ser aprobada. Para evitar la división entre 0 se devuelve directamente un umbral muy alto (el máximo entero). Salvada esta restricción, dada la fórmula en el enunciado, para poder representar el 0,2 en aritmética de enteros se ha optado por multiplicar los factores por potencias de 10 y, para no perder demasiada precisión, se ha modificado la ecuación para hacer una equivalente que retrase lo máximo posible la división entre el presupuesto total. Debe tenerse en cuenta que la variable p.presupuesto puede ser establecida por cualquier usuario sin necesidad de transferencia por lo que podría ser un número muy elevado si quiere, de alguna manera, causar algún problema al contrato. No es posible que haya overflow pero en un caso muy límite podría darse que nunca se aceptase la propuesta por error en el cálculo del umbral, lo cual no pararía el funcionamiento del sistema en cualquier caso.

De superar el umbral y tener el presupuesto suficiente para aprobarla, se marca como aprobada y se añade al array de aprobadas (quitándolo a su vez del array de propuestas de financiación pendientes), se hacen las modificaciones en el presupuesto y se decrementa el número de propuestas pendientes. Una vez aprobada la propuesta, se queman los tokens asociados a ella y se llama a su función de ejecución.

Esta llamada al execute de cierta propuesta hace que QuadraticVoting pierda el control de la ejecución, lo que puede ser perjudicial para el contrato si la función a la que se llama resulta ser maliciosa. Al ser una función interna, no es posible que se la llame desde fuera por lo que la única manera de reentrar a la función sería llamando de nuevo a stake que es el único punto del contrato desde donde se llama a checkAndExecuteProposal pero no es posible reentrar desde la llamada al execute porque el lock de aprobada ha sido modificado a true y este se va a volver a comprobar en la función stake. Otro flujo que podría aprovecharse sería hacer un ataque de reentrada combinado y llamar desde el execute a la función cancelProposal para obtener el beneficio del presupuesto que se le pasa al contrato de la propuesta aprobada y además conseguir el asociado a reclamar los tokens de una propuesta cancelada. Se ve que esto de nuevo no es posible al modificar aprobada antes de ejecutar execute, esta variable será comprobada de nuevo en cancelProposal haciendo que sea imposible cancelar la propuesta. Tampoco es posible hacerse con el control de los tokens usados para emitir votos a la propuesta que se aprueba de otra manera al hacer el burn antes de la llamada a execute.

1.1.14. withdrawFromProposal

```

1 //Retirar los votos del participante de cierta propuesta
2 function withdrawFromProposal (uint id, uint cantidad) public estaActivo()
   estaAbierta() {
3
4     Propuesta storage p = _propuestas[id];
5     //solo puede retirar votos de una propuesta no aprobada y sus propios
       votos
6     require(!p.aprobada, "Propuesta ya aprobada");
7     uint votos = votoPorVotante[id][msg.sender];
8     require(votos >= cantidad, "No tienes votos suficientes en esta propuesta"
       );
9
10    //debe pertenecer a esta ronda para poder retirar votos
11    require(p.ronda == _ronda, "La propuesta no pertenece a esta ronda");
12
13    //cantidad a devolver siguiendo la logica cuadratica
14    uint votosRestantes = votos - cantidad;
15    uint tokensADevolver = (votos * votos) - (votosRestantes * votosRestantes)
       ;
16
17    //actualizar estado
18    votoPorVotante[id][msg.sender] -= cantidad;
19    p.numVotos -= cantidad;
20    p.numTokens -= tokensADevolver;
21

```

```

22     //devolver tokens
23     bool success = _contratoERC20.transferFrom(address(this), msg.sender ,
24         tokensADevolver);
25     require(success, "Fallo al transferir tokens");
    }

```

Listing 15: Función withdrawFromProposal

Permite a un participante inscrito, con la votación abierta, retirar un número de votos de cierta propuesta dado su id y el número de votos a retirar. Se comprueba que la propuesta no ha sido aprobada, que pertenece a la ronda actual y que el participante ha emitido al menos el número de votos que quiere retirar a la propuesta. Usando el sistema cuadrático, se calcula el número de tokens que deben devolverse y se modifican los datos de la propuesta acorde a los votos retirados(todo lo anterior sabiendo que no puede haber errores de overflow por la versión del compilador). Por último, se devuelve la propiedad de los tokens al participante haciendo uso del contrato de gestión de tokens y la función transferFrom.

1.1.15. closeVoting y reiniciarVotacion

```

1     //Cierre de la votacion solo por el propietario
2     function closeVoting () public onlyOwner() estaAbierta() {
3         _estadoVotacion = 2; //se pone en estado intermedio
4     }
5
6     function reiniciarVotacion() public onlyOwner() estadoIntermedio(){
7         if (_presupuesto > 0) {
8             uint restante = _presupuesto;
9             _presupuesto = 0; //antes para evitar reentrada
10            (bool success, ) = _propietario.call{value: restante}("");
11            require(success, "Error al devolver Ether");
12        }
13        _estadoVotacion = 0; //pasa a estar cerrado
14        _numPendingProposals = 0;
15        delete _propuestasAprobadas;
16        delete _propuestasFinanciacion;
17        delete _propuestasSignaling;
18    }

```

Listing 16: Funciones closeVoting y reiniciarVotacion

Con el objetivo de implementar el patrón pull-over-push de la mejor forma posible y para hacer el sistema de rondas, se implementa la función de closeVoting que solo cambia el estado de la votación a uno intermedio entre el cierre de una ronda y la apertura de otra. Este estado intermedio permite reclamar los tokens de propuestas no aprobadas de la ronda actual y ejecutar las propuestas signaling de la ronda actual una vez termine y pase al estado intermedio. Dado que se pueden reclamar tokens de propuestas no aprobadas de rondas anteriores en cualquier momento (análogo para la ejecución de propuestas signaling) se ha implementado un cierre en dos fases y el estado intermedio para poder hacer estas tareas con las propuestas de la ronda actual una vez se cierre con closeVoting pero sin necesidad de que se inicie una ronda nueva ya que puede no iniciarse nunca y no se podrían reclamar tokens o ejecutar propuestas de la ronda que acaba de terminar.

Cuando se desee abrir una nueva ronda de voto, es necesario reiniciar con la función implementada que devuelve el presupuesto restante al propietario y reinicia ciertos atributos para comenzar una nueva ronda. El array de las propuestas canceladas no se reinicia para poder ser consultado en cualquier momento y que los participantes accedan a los ids de las propuestas canceladas en cualquier ronda posterior desde que se cancelo cierta propuesta, puesto que los tokens pueden reclamarse cuando se quiera.

Especial atención hay que prestar a la transferencia plana de Ether que puede hacerse a un contrato malicioso que quiera atacar al sistema. Se usa la variable restante para poder modificar la variable presupuesto

a 0 antes de la llamada y evitar que el atacante se beneficie de hacer múltiples llamadas a esta función obteniendo más Ether del que le corresponde. En caso de fallar en la transferencia, se revierte la ejecución de la función.

1.1.16. reclamarTokens

```
1 //reclamar los tokens cuando ha terminado la votacion (bajo demanda siguiendo
  el patron)
2 function reclamarTokens(uint id) public estaInscrito(){
3     Propuesta storage p = _propuestas[id];
4     require(!p.aprobada, "Propuesta aprobada");
5
6     uint votos = votoPorVotante[id][msg.sender];
7     require(votos > 0, "No tienes votos en esta propuesta");
8
9     require(p.ronda < _ronda || (p.ronda == _ronda && _estadoVotacion ==2), "
      Propuesta de ronda no valida");
10
11     uint tokensADevolver = votos * votos;
12     votoPorVotante[id][msg.sender] = 0; //antes para evitar reentrada
13     p.numVotos -=votos;
14     p.numTokens -=tokensADevolver;
15
16     bool success = _contratoERC20.transferFrom(address(this), msg.sender,
      tokensADevolver);
17     require(success, "Fallo al transferir tokens");
18     if(p.numVotos == 0) delete _propuestas[id];
19 }
```

Listing 17: Función reclamarTokens

Se trata de una función implementada como parte del patrón opcional. En la primera implementación del patrón, se permitía reclamar los tokens de aquellas propuestas que habían quedado pendientes una vez finalizada la votación. Con el sistema de rondas final, se permite reclamar tokens de propuestas que quedaron pendientes en rondas anteriores y también reclamar tokens de propuestas de la ronda actual estando en el estado intermedio. Se calculan los tokens que deben ser devueltos y se actualizan los datos de la propuesta respecto a eso, sabiendo que no ocurrirán errores por overflow. Finalmente, se devuelve la propiedad de los tokens al participante y, en caso de que no queden votos en la propuesta y ningún participante vaya reclamar tokens en ella, se elimina la propuesta del mapa para liberar espacio.

1.1.17. ejecutarPropuestaSignaling

```
1 //ejecutar las propuestas signaling bajo demanda
2 function ejecutarPropuestaSignaling(uint id) public {
3     Propuesta storage p = _propuestas[id];
4     require(!p.aprobada, "Propuesta ya aprobada");
5     require(p.presupuesto == 0, "No es signaling");
6     require(p.creador == msg.sender, "Solo el creador puede ejecutarla");
7     require(p.ronda < _ronda || (p.ronda == _ronda && _estadoVotacion ==2), "
      Propuesta de ronda no valida");
8
9     p.aprobada = true;
10
11     //quemar los tokens
12     _contratoERC20.burnToken(address(this), p.numTokens);
13
14     (bool success, ) = p.contrato.call{gas: 100000}({
```



```

15         abi.encodeWithSignature("executeProposal(uint256,uint256,uint256)", id
16             , p.numVotos, p.numTokens)
17     );
18     require(success, "Fallo al ejecutar signaling");
19 }

```

Listing 18: Función ejecutarPropuestaSignaling

Otra función que sigue el patrón pull-over-push que permite al creador de una propuesta signaling ejecutarla. Esta ejecución se dará en cualquier momento si la propuesta pertenece a una ronda previa o en el estado intermedio si la propuesta pertenece a la ronda actual. Con las comprobaciones hechas, se cambia el estado de la propuesta a aprobada y se ejecuta la función executeProposal del contrato de la propuesta.

En este caso, vuelve a aparecer una llamada a una función externa que puede ser maliciosa. Del mismo modo que con las propuestas de financiación, se cambia el lock de propuesta aprobada a true y se queman los tokens usados en la votación de la propuesta antes de hacer la llamada externa, evitando caer en ataques.

1.2. VoteToken

```

1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity ^0.8.0;
3
4  import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5
6  contract VoteToken is ERC20{
7      address public admin;
8
9      constructor(address contrato, string memory nombre, string memory simbolo)
10         ERC20(nombre,simbolo){
11         admin=contrato; //el admin es el contrato
12     }
13
14     modifier onlyAdmin() {
15         require(msg.sender==admin, "Solo el administrador puede hacer esto");
16         _;
17     }
18
19     //creacion de tokens
20     function createToken(address destinatario, uint256 cantidad) external
21         onlyAdmin{
22         _mint(destinatario, cantidad); //solo el admin puede crear tokens???
23     }
24
25     //eliminar tokens
26     function burnToken(address cuenta, uint256 cantidad) external onlyAdmin{
27         _burn(cuenta,cantidad); //aqui si creo que solo el admin puede eliminar
28         tokens
29     }
30 }

```

Listing 19: voteToken.sol

Es el contrato de gestión de los tokens ERC20 usados en el sistema de votación. Se usa la implementación de Open Zeppelin y se consigue que el administrador del contrato sea el propio contrato QuadraticVoting para evitar que cualquier dirección pueda crear o quemar tokens a su antojo.

1.3. IExecutableProposal

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.0;
3
4 interface IERC165 {
5     function supportsInterface(bytes4 interfaceId) external view returns (bool);
6 }
7
8 interface IExecutableProposal is IERC165 {
9     function executeProposal(uint proposalId, uint numVotes, uint numTokens)
10         external payable;
11 }
```

Listing 20: IExecutableProposal.sol

Interfaz que deben implementar los contratos de propuestas para que tengan la función de ejecución correspondiente. Se ha utilizado la interfaz IERC165 con su función supportsInterface para comprobar que los contratos asociados a propuestas implementan la interfaz IExecutableProposal necesaria.

2. Pruebas

Ejecutamos las funciones en conjunto para probar que se realicen correctamente

2.1. Ejecución de cada función

2.1.1. Creación del contrato QuadraticVoting

En la creación de este contrato se debe proporcionar el precio en Wei de cada token y el número máximo de tokens que se van a poner a la venta para votaciones. Se inicializan los atributos correspondientes y se crea el contrato ERC20 que gestiona los tokens.

ACCOUNT +  

0x5B3...eddC4 (99.999999999... 

GAS LIMIT

☒ Estimated Gas

☐ Custom 3000000

VALUE

0 Wei 

CONTRACT

QuadraticVoting - contracts/QuadraticVoting 

evm version: cancun

DEPLOY 

precioToken: 1

maxTokens: 15

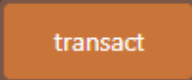
 Calldata  Parameters 

Figura 1: Parámetros del constructor

Deployed Contracts 1 

✓ QUADRATICVOTING AT 0XD41   

Balance: 0 ETH



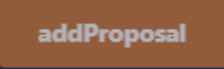
 string titulo, string descr 

Figura 2: El contrato se ha desplegado

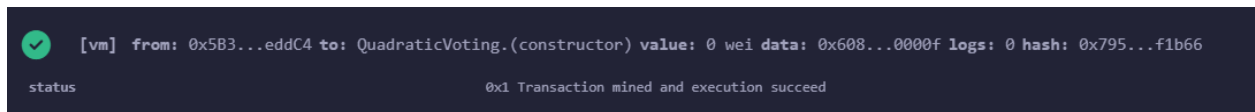


Figura 3: La transacción se ha realizado con éxito

2.1.2. Apertura de la votación

Para abrir la votación se requiere que no haya una votación en curso. Solo lo puede ejecutar el usuario que ha creado el contrato. En la transacción que ejecuta esta función se debe transferir el presupuesto inicial del que se va a disponer para financiar propuestas.

Al tratar de abrir la votación desde otra cuenta distinta, se nos informa que no se es el propietario del contrato y se revierte la ejecución.

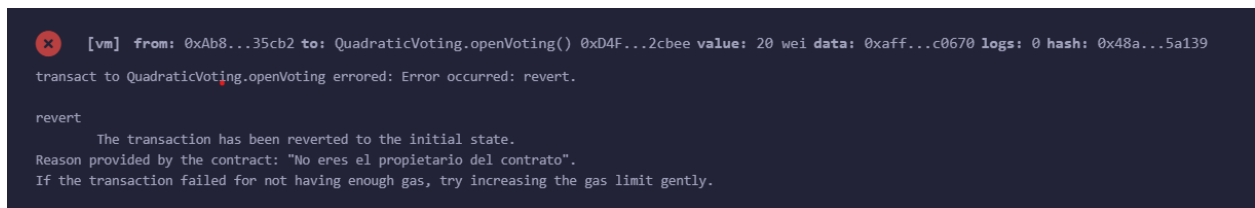


Figura 4: Apertura de la votación con cuenta diferente

El propietario ha podido abrir correctamente la votación aportando un presupuesto inicial de 100 wei.



Figura 5: Apertura de la votación por parte del propietario

Al tratar de volver a abrir la votación, se le informa al usuario que la votación no ha sido cerrada y se revierte la ejecución.

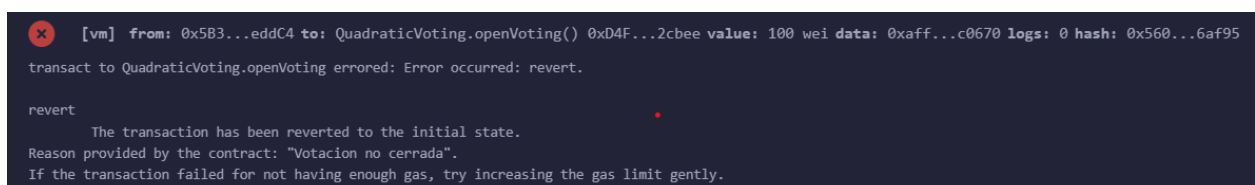


Figura 6: Apertura en una votación abierta

2.1.3. Añadir un participante

En el caso en que al añadir un participante se proporcione una cantidad menor al precio de un token, la ejecución se revierte y se informa al usuario

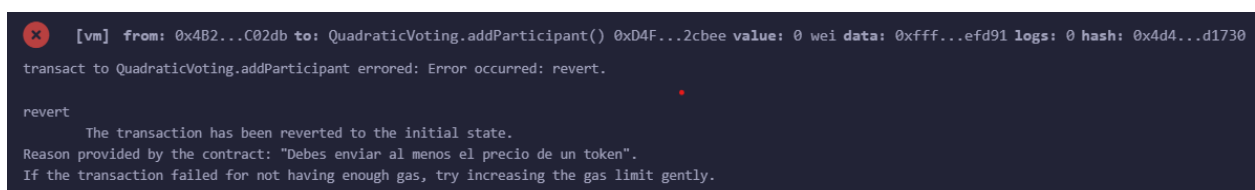


Figura 7: Añadir un participante sin enviar una cantidad suficiente

Se ha añadido un participante aportando una cantidad de 3 wei, con lo que pasa a poseer 3 tokens.

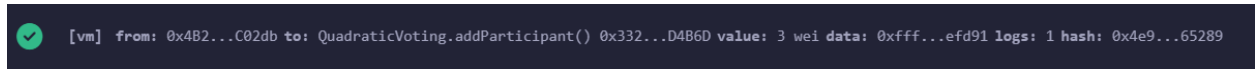


Figura 8: Enter Caption



Figura 9: Obtener dirección del contrato ERC20

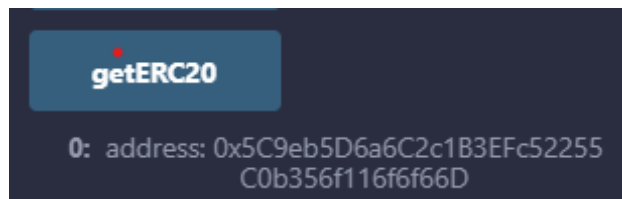


Figura 10: Dirección del contrato ERC20

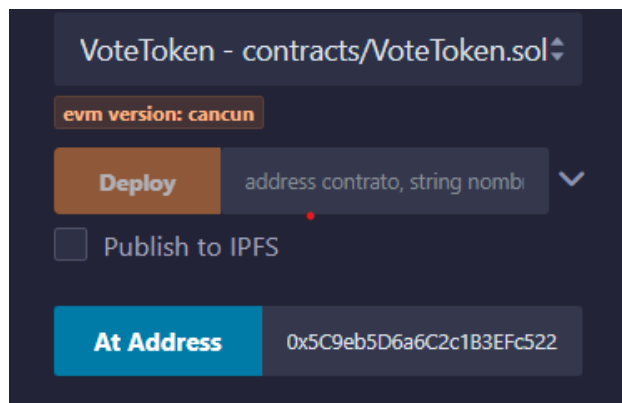


Figura 11: VoteToken con la dirección del contrato

Comprobamos que el participante 1 efectivamente tiene 3 tokens

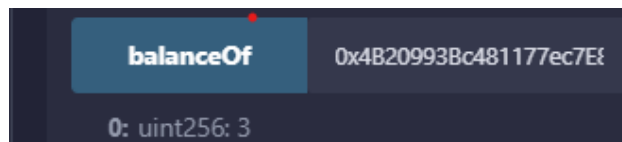


Figura 12: Balance del Participante 1

Al tratar de volver a añadir al participante, se le informa de que ya está inscrito y se revierte la ejecución

```

[vm] from: 0x4B2...C02db to: QuadraticVoting.addParticipant() 0x332...D4B6D value: 1 wei data: 0xfff...efd91 logs: 0 hash: 0xa76...e733a
transact to QuadraticVoting.addParticipant errored: Error occurred: revert.

revert
  The transaction has been reverted to the initial state.
Reason provided by the contract: "Ya estas activo".
If the transaction failed for not having enough gas, try increasing the gas limit gently.

```

Figura 13: Participante ya inscrito

_estadoVotaci...	
0: uint256: 1	
_inscritos	0x4B20993Bc481177ec7E8
0: bool: true	
_maxTokens	
0: uint256: 15	
_numParticip...	
0: uint256: 1	
_numPending...	
0: uint256: 0	
_numTokens	
0: uint256: 3	
_precioToken	
0: uint256: 1	
_presupuesto	
0: uint256: 100	
_propietario	

Figura 14: Estado actual del sistema de votación

Añadimos otro participante distinto:

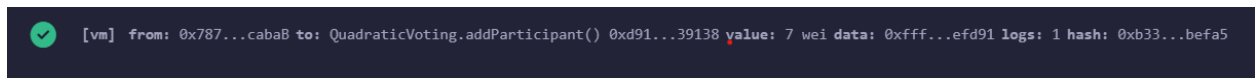


Figura 15: Enter Caption

2.1.4. Eliminar un participante



Figura 16: El propio participante se elimina

Al tratar de eliminar una dirección que no es participante del sistema de votación, se revierte la ejecución e informa

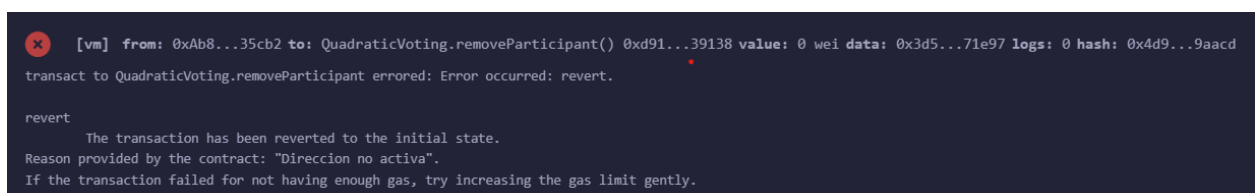


Figura 17: Eliminar no participante

2.1.5. Añadir una propuesta

Volvemos a inscribir al Participante1 y vamos a añadir una propuesta. Primero desplegamos un contrato de ejemplo de propuesta y después la añadimos con sus correspondientes datos.

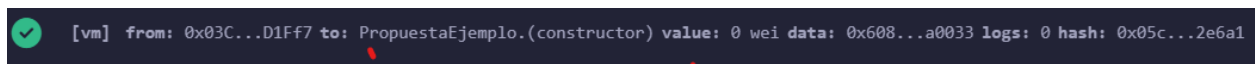


Figura 18: Propuesta1 desplegada

Figura 19: Propuesta1

```
[vm] from: 0x482...C02db to: QuadraticVoting.addProposal(string,string,uint256,address) 0xd91...39138 value: 0 wei data: 0x9ab...00000 logs: 0  
hash: 0xcd6...35958
```

Figura 20: Transacción exitosa

Añadimos también una propuesta de signaling:

The screenshot shows a web interface titled "ADDPROPOSAL". It contains four input fields: "titulo:" with the value "Propuesta2", "descripcion:" with the value "Propuesta2", "presupuesto:" with the value "0", and "contrato:" with the value "0x2367298FFE28a551DDBa033ed". Below the fields are three buttons: "Calldata", "Parameters", and "transact". The "transact" button is highlighted in orange.

Figura 21: Propuesta2

Ahora añadimos una propuesta de financiación con el Participante2:

The screenshot shows a web interface titled "ADDPROPOSAL". It contains four input fields: "titulo:" with the value "Propuesta3", "descripcion:" with the value "Propuesta3", "presupuesto:" with the value "3", and "contrato:" with the value "0xBF85582f17e04E7E37EeEBBd8e". Below the fields are three buttons: "Calldata", "Parameters", and "transact". The "transact" button is highlighted in orange.

Figura 22: Propuesta3

```
[vm] from: 0x787...cabaB to: QuadraticVoting.addProposal(string,string,uint256,address) 0xd91...39138 value: 0 wei data: 0x9ab...00000 logs: 0  
hash: 0x57f...06e20
```

Figura 23: Transacción exitosa

2.1.6. Obtener propuestas de financiación pendientes

[illegible]

Figura 24: Propuestas pendientes

2.1.7. Obtener propuestas de signaling

[illegible]

Figura 25: Propuestas de signaling

2.1.8. Sell tokens

El Participante1 quiere vender sus tokens (devolvérselos al contrato de votaciones). Para poder hacerlo, debe aprobar su dirección y la cantidad de tokens.

✓ [vm] from: 0x4B2...C02db to: VoteToken.approve(address,uint256) 0x5C9...6f66D value: 0 wei data: 0x095...00003 logs: 1 hash: 0x57c...65a56

Figura 26: Approve

✓ [vm] from: 0x4B2...C02db to: QuadraticVoting.sellTokens(uint256) 0xd91...39138 value: 0 wei data: 0x6c1...00003 logs: 1 hash: 0x4e2...512eb

Figura 27: Venta exitosa

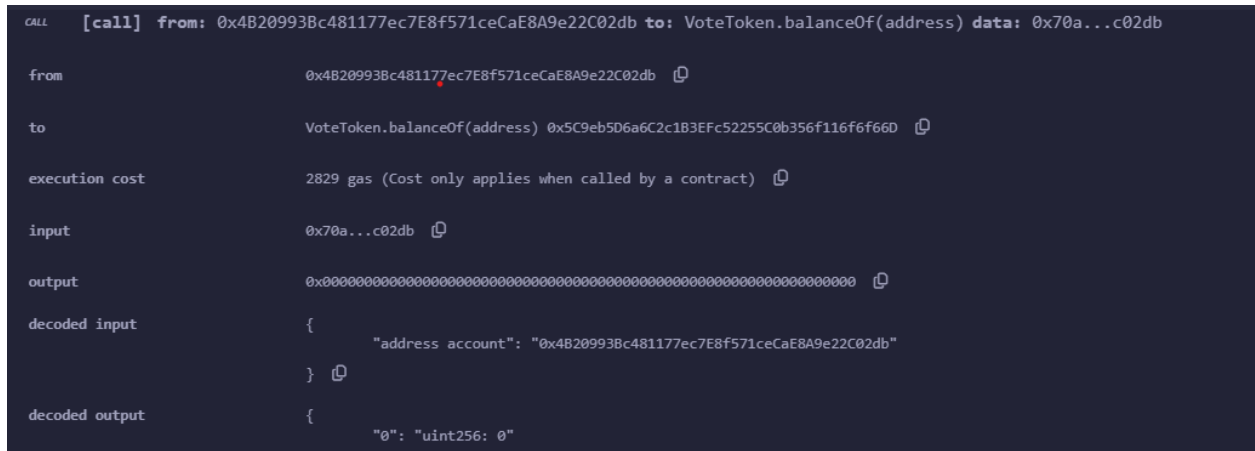


Figura 28: El balance de Participante1 es 0

El número de tokens de la votación pasa a ser 3

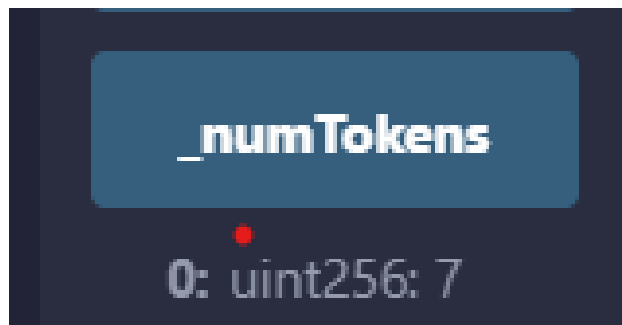


Figura 29: NumTokens

2.1.9. Buy tokens

El participante2 quiere comprar 3 tokens:

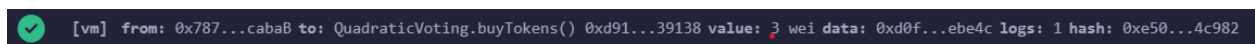


Figura 30: Transacción exitosa

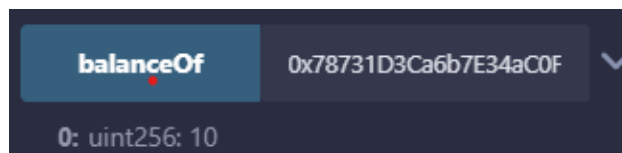


Figura 31: Su balance pasa a ser 10

El número total de tokens de la votación vuelve a ser 10


```
✓ [vm] from: 0x787...cabaB to: QuadraticVoting.stake(uint256,uint256) 0xd91...39138 value: 0 wei data: 0x7b0...00003 logs: 1 hash: 0x750...f1e25
```

Figura 37: Transacción exitosa

The image shows a web interface titled "STAKE". It has two input fields: "id:" with the value "3" and "nuevosVotos:" with the value "3". Below these fields are three buttons: "Calldata", "Parameters", and "transact". The "transact" button is orange and is the most prominent.

Figura 36: Deposita 3 votos

Al depositar 3 votos, han sido utilizados 9 tokens, por lo que se queda 1 token

```
CALL [call] from: 0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB to: VoteToken.balanceOf(address) data: 0x70a...cabab

from 0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB
to VoteToken.balanceOf(address) 0x5C9eb5D6a6C2c1B3EFc52255C0b356f116f6f66D
execution cost 2829 gas (Cost only applies when called by a contract)
input 0x70a...cabab
output 0x0000000000000000000000000000000000000000000000000000000000000001
decoded input {
  "address account": "0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB"
}
decoded output {
  "0": "uint256: 1"
```

Figura 38: Balance Participante3

_propuestas	3
0: uint256: presupuesto	3
1: uint256: numVotos	3
2: uint256: numTokens	9
3: uint256: posicion_array	1
4: uint256: ronda	1
5: address: contrato	0xBF85582f17e04E7E37EeEBBd8e7c993587F58932
6: address: creador	0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB
7: bool: aprobada	true
8: bool: cancelada	false
9: string: titulo	Propuesta3
10: string: descripcion	Propuesta3

Figura 39: La propuesta ha sido aprobada

Ahora el contrato de la Propuesta3 tiene un saldo de 3 eth, equivalente a los tokens correspondientes a la votación de la propuesta



Figura 40: Balance del contrato de la Propuesta3

2.1.12. Obtener propuestas aprobadas

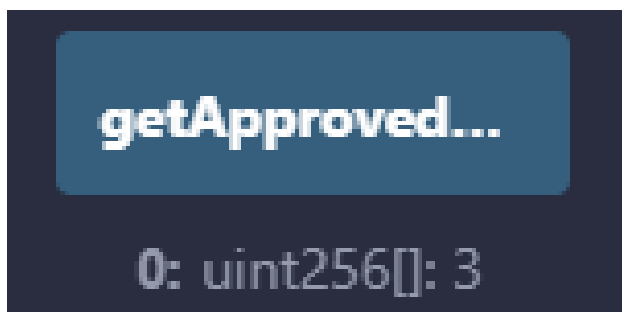


Figura 41: Aprobadas

El participante1 ha comprado tokens y ahora tiene 3. Deposita 1 voto en la propuesta1

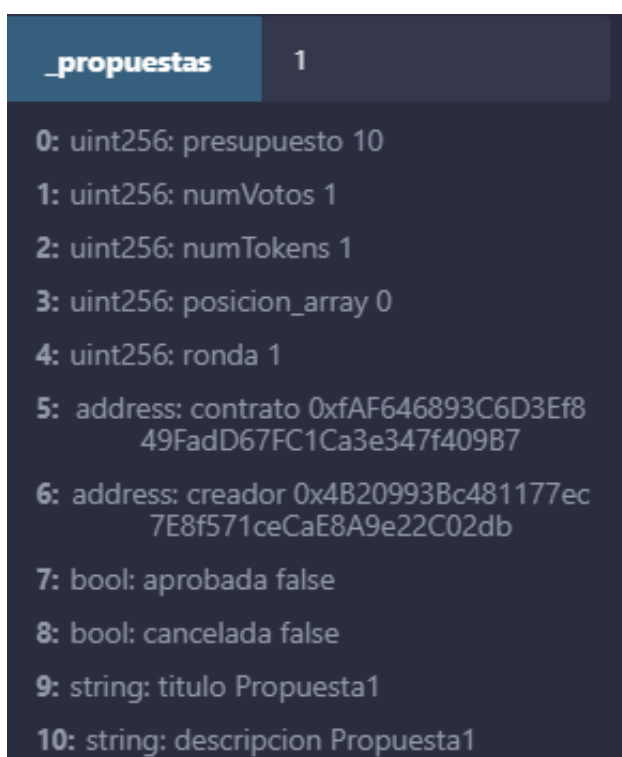


Figura 42: Propuesta1

Ahora cancela



Figura 43: Cancela la propuesta1

2.1.13. Obtener propuestas canceladas

[illegible]

Figura 44: Canceladas

2.1.14. Reclamar tokens de propuestas canceladas

✓ [vm] from: 0x482...C02db to: QuadraticVoting.reclamarTokensCanceladas(uint256) 0xd91...39138 value: 0 wei data: 0x781...00001 logs: 1 hash: 0x4a1...e9a61

Figura 45: Transacción ejecutada

Vuelve a tener 3 tokens (ha recuperado 1)

```
CALL [call] from: 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db to: VoteToken.balanceOf(address) data: 0x70a...c02db

from 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db ⓘ

to VoteToken.balanceOf(address) 0x5C9eb5D6a6C2c1B3EFc52255C0b356f116f6f66D ⓘ

execution cost 2829 gas (Cost only applies when called by a contract) ⓘ

input 0x70a...c02db ⓘ

output 0x0000000000000000000000000000000000000000000000000000000000000000 ⓘ

decoded input {
  "address account": "0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db"
} ⓘ

decoded output {
  "0": "uint256: 3"
```

Figura 46: Balance Participante1

2.1.15. Cerrar la votación

✓ [vm] from: 0x5B3...eddC4 to: QuadraticVoting.closeVoting() 0xd91...39138 value: 0 wei data: 0xc63...1b292 logs: 0 hash: 0x375...50c3f

Figura 47: Votación cerrada.

2.1.16. Ejecutar propuestas signaling

```
[vm] from: 0x4B2...C02db to: QuadraticVoting.ejecutarPropuestaSignaling(uint256) 0xd91...39138 value: 0 wei data: 0xaa2...00002 logs: 0  
hash: 0x067...55854
```

Figura 48: Transacción ejecutada

2.1.17. Retirar tokens de una propuesta

Partiendo de la base de que Participante1 tiene un voto en Propuesta1, retira 1 token de la propuesta1.

```
[vm] from: 0x4B2...C02db to: QuadraticVoting.withdrawFromProposal(uint256,uint256) 0xd91...39138 value: 0 wei data: 0x649...00001 logs: 1  
hash: 0x7fc...96de7
```

Figura 49: Transacción exitosa

Participante1 vuelve a tener 3 tokens (ha recuperado 1)

```
CALL [call] from: 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db to: VoteToken.balanceOf(address) data: 0x70a...c02db  
  
from 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db  
  
to VoteToken.balanceOf(address) 0x5C9eb5D6a6C2c183EFc52255C0b356f116f6f66D  
  
execution cost 2829 gas (Cost only applies when called by a contract)  
  
input 0x70a...c02db  
  
output 0x0000000000000000000000000000000000000000000000000000000000000003  
  
decoded input {  
  "address account": "0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db"  
}  
  
decoded output {  
  "0": "uint256: 3"}
```

Figura 50: Enter Caption

Partiendo de la base de que Participante1 tiene un voto en Propuesta1, retira 1 token de la propuesta1.

2.1.18. Reclamar tokens

Partiendo de la base de que Participante1 tenía 1 token. Ahora tiene un voto en Propuesta1 y la votación se ha cerrado. Reclama 1 token de la propuesta1.

```
[vm] from: 0x4B2...C02db to: QuadraticVoting.reclamarTokens(uint256) 0xd91...39138 value: 0 wei data: 0xcb7...00001 logs: 1 hash: 0xdf6...ae984
```

Figura 51: Transacción exitosa

Después de abrirse otra votación, Participante1 reclama sus tokens de otra ronda de la Propuesta1

```
[vm] from: 0x4B2...C02db to: QuadraticVoting.reclamarTokens(uint256) 0xd91...39138 value: 0 wei data: 0xcb7...00002 logs: 1 hash: 0x9dc...a99bb
```

Figura 52: Transacción exitosa